

EV Charging Station Analysis

Complete Python Pandas Solution

This document contains the complete solution for all six operations of the EV Charging Station Analysis task.

1. Import Pandas and Create Class

```
import pandas as pd

class EVChargingAnalyzer:
```

2. Operation 1: Create Charging Sessions DataFrame

```
def create_sessions_df(self, session_data: list) -> pd.DataFrame:
    columns = [
        "SessionID",
        "StationID",
        "City",
        "ChargingDate",
        "EnergyKWh",
        "DurationMinutes",
        "PaymentStatus"
    ]

    df = pd.DataFrame(session_data, columns=columns)

    return df
```

3. Operation 2: Clean Charging Sessions

```
def clean_sessions_data(self, df: pd.DataFrame) -> pd.DataFrame:
    valid_status = ["Paid", "Pending", "Failed"]

    df = df.dropna(
        subset=["SessionID", "StationID", "City", "PaymentStatus"]
    )

    df = df[
        (df["EnergyKWh"] > 0) &
        (df["DurationMinutes"] > 0) &
        (df["PaymentStatus"].isin(valid_status))
    ]

    return df
```

4. Operation 3: Add Long Session Flag

```
def add_long_session_flag(
    self,
    df: pd.DataFrame,
    duration_threshold: int
) -> pd.DataFrame:

    df["IsLongSession"] = (
        df["DurationMinutes"] > duration_threshold
    ).astype(int)

    return df
```

5. Operation 4: Station Utilization Summary

```
def station_utilization_summary(
    self,
    df: pd.DataFrame
) -> pd.DataFrame:

    result = df.groupby("StationID").agg(
        SessionCount=("SessionID", "count"),
        TotalEnergyKWh=("EnergyKWh", "sum"),
        AverageDuration=("DurationMinutes", "mean")
    ).reset_index()

    result["AverageDuration"] = result["AverageDuration"].round(1)
```

```
return result
```

6. Operation 5: High Energy Stations

```
def high_energy_stations(
    self,
    df: pd.DataFrame,
    energy_threshold: float
) -> pd.DataFrame:

    result = df.groupby("StationID").agg(
        TotalEnergyKWh=("EnergyKWh", "sum")
    ).reset_index()

    result = result[
        result["TotalEnergyKWh"] > energy_threshold
    ]

    return result
```

7. Operation 6: City Revenue Summary

```
def city_revenue_summary(
    self,
    df: pd.DataFrame
) -> pd.DataFrame:

    paid_df = df[df["PaymentStatus"] == "Paid"].copy()

    paid_df["Revenue"] = paid_df["EnergyKWh"] * 18

    result = paid_df.groupby("City").agg(
        Revenue=("Revenue", "sum")
    ).reset_index()

    return result
```

Complete Code for solution.py

```
import pandas as pd

class EVChargingAnalyzer:

    # Operation 1: Create Charging Sessions DataFrame
    def create_sessions_df(self, session_data: list) -> pd.DataFrame:

        columns = [
            "SessionID",
            "StationID",
            "City",
            "ChargingDate",
            "EnergyKWh",
            "DurationMinutes",
            "PaymentStatus"
        ]

        df = pd.DataFrame(session_data, columns=columns)

        return df

    # Operation 2: Clean Charging Sessions
    def clean_sessions_data(self, df: pd.DataFrame) -> pd.DataFrame:

        valid_status = ["Paid", "Pending", "Failed"]

        df = df.dropna(
            subset=[
                "SessionID",
                "StationID",
                "City",
                "PaymentStatus"
            ]
        )

        df = df[
            (df["EnergyKWh"] > 0) &
            (df["DurationMinutes"] > 0) &
            (df["PaymentStatus"].isin(valid_status))
        ]

        return df

    # Operation 3: Add Long Session Flag
    def add_long_session_flag(
        self,
        df: pd.DataFrame,
        duration_threshold: int
    ) -> pd.DataFrame:

        df["IsLongSession"] = (
            df["DurationMinutes"] > duration_threshold
        ).astype(int)

        return df

    # Operation 4: Station Utilization Summary
    def station_utilization_summary(
        self,
        df: pd.DataFrame
    ) -> pd.DataFrame:

        result = df.groupby("StationID").agg(
            SessionCount=("SessionID", "count"),
            TotalEnergyKWh=("EnergyKWh", "sum"),
            AverageDuration=("DurationMinutes", "mean")
        ).reset_index()

        result["AverageDuration"] = result["AverageDuration"].round(1)

        return result

    # Operation 5: High Energy Stations
    def high_energy_stations(
        self,
        df: pd.DataFrame,
        energy_threshold: float
    ) -> pd.DataFrame:
```

```

        result = df.groupby("StationID").agg(
            TotalEnergyKWh=("EnergyKWh", "sum")
        ).reset_index()

        result = result[
            result["TotalEnergyKWh"] > energy_threshold
        ]

    return result

# Operation 6: City Revenue Summary
def city_revenue_summary(
    self,
    df: pd.DataFrame
) -> pd.DataFrame:

    paid_df = df[df["PaymentStatus"] == "Paid"].copy()

    paid_df["Revenue"] = paid_df["EnergyKWh"] * 18

    result = paid_df.groupby("City").agg(
        Revenue=("Revenue", "sum")
    ).reset_index()

    return result

```